



University
of Basel

Chapter 5.2: Parallel Architectures and Programming Models

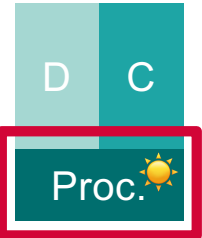
Foundations of Distributed Systems

Fall Semester 2025

Prof. Dr. Florina Ciorba

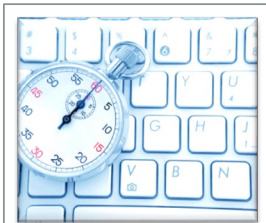


Addressed in The **Processing** Lecture Series



- Goals

Chapter 5.2:
Parallel architectures & models



Performance

```
00111101001010101
01001010100010101
01010ERROR011010
10100010100110001
01100010100100111
10100010100110001
```

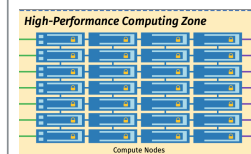
Resiliency



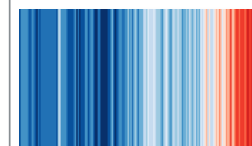
Extreme scalability



Energy



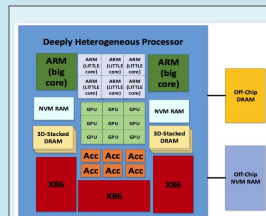
Security



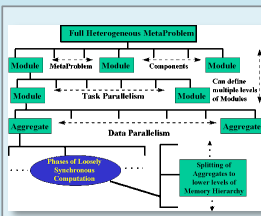
Sustainability



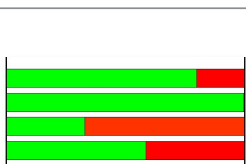
Convergence
HPC+Data+AI



Heterogeneity



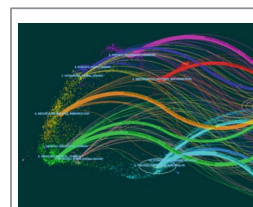
Multilevel parallelism



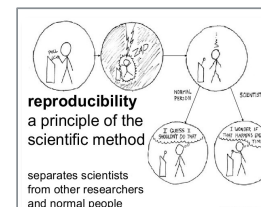
Load imbalance



Programming



Visualization



Reproducibility

Overview of Chapter 5.2: Parallel Architectures & Programming Models

-
- | | |
|---|-----------------------------|
| a | Memory Architectures |
| b | Parallel Programming Models |
| c | Network Topologies |
-

Learning objectives

Understand parallel computer architectures, including memory organization and network interconnections.

Understand their role in defining parallel programming models.

Parallel Computers and Their Programming

What We Learned So Far

A. Memory architectures

Machine model 1: Shared memory

⇒ Symmetric multiprocessing & UMA

Machine model 2: Distributed memory

⇒ Asymmetric multiprocessing & NUMA

Machine model 3: Hybrid distributed shared memory (combines 1+2)

⇒ Asymmetric multiprocessing & NUMA, GAS

Machine model 4: Distributed shared memory (variation of 3)

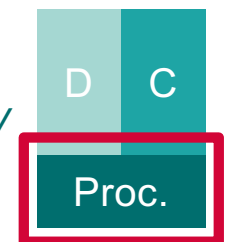
⇒ Asymmetric multiprocessing & NUMA, DGAS

Machine model 5: Fully distributed systems (not parallel, not covered by)

B. Parallel programming models

C. Network topologies

What the machine offers
w.r.t. memory



Parallel Computers and Their Programming

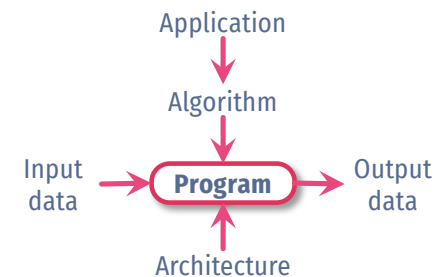
As Easy as A, B, C

A. Memory architectures

B. Parallel programming models

What the programmer sees
w.r.t. memory

- **Abstraction** above hardware and memory architectures, offering a view of **data** (where stored, how organized) & **execution** (what the program does)
- **Programs = algorithms + data structuring** (+ hardware view)*
 - Where **architecture** meets **applications** and vice versa
 - 🤝 Cannot write **parallel** programs *without a faint idea* of the potential **parallel** system
- Independence Benefits
 - **Application** $\leftrightarrow$ **Architecture**: **write once**, run on many architectures
 - **Architecture** $\leftrightarrow$ **Applications**: **design once**, execute many applications



C. Network topologies

“Parallel programs are difficult to write: performance is affected by data dependencies, race conditions, synchronization, and parallel slowdown.”
 (“Energy-Efficient Distributed Computing Systems | Wiley”, 2012, p. 499)

Parallel Computers and Their Programming

As Easy as A, B, C

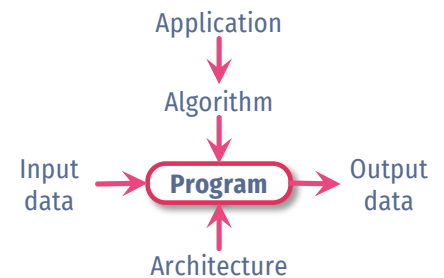
A. Memory architectures

B. Parallel programming models (use explicit APIs)

1. Shared memory (**without** threads)
2. Shared memory (**with** threads)
3. Distributed memory / Message Passing
4. Data parallel
5. Hybrid distributed shared memory
6. Single Program Multiple Data (SPMD)
7. Multiple Program Multiple Data (MPMD)

C. Network topologies

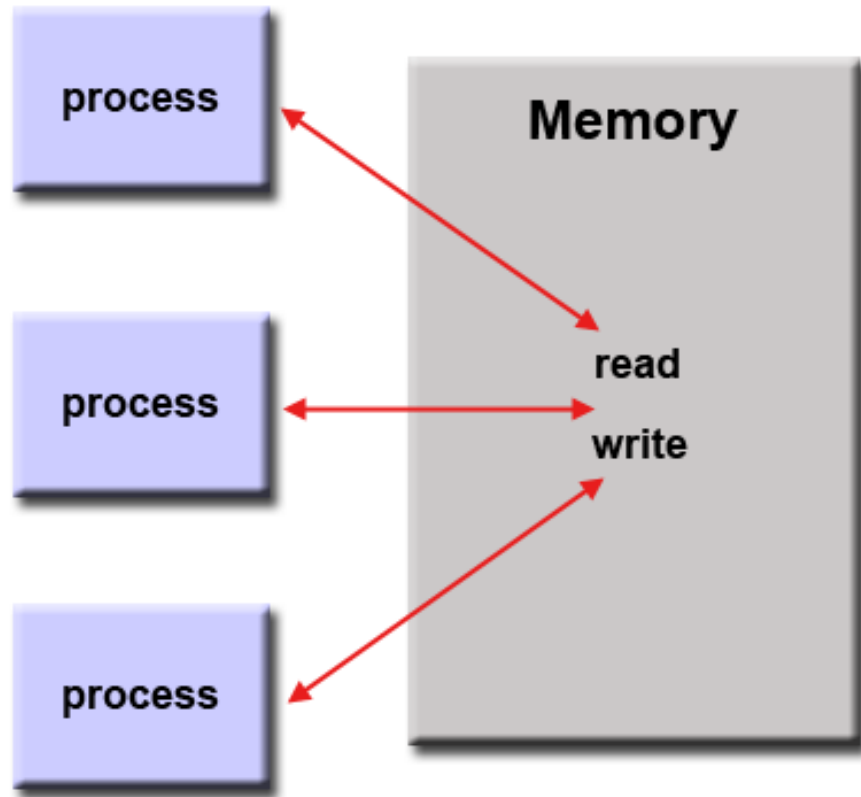
What the programmer sees
w.r.t. memory



*“... these models are **not** specific to a particular type of machine or memory architecture. [...] any of these models can (theoretically) be implemented on any underlying hardware”*

[https://computing.llnl.gov/tutorials/parallel_comp]

Programming Model 1: Shared memory (without threads)



- Processes **share** a **common** memory address space, with **full access** to *read* from and *write* to **asynchronously**
- Locks/semaphores** are used to **control concurrent access** to the **shared memory**, resolve contention, prevent race conditions and deadlocks

Advantage: No “data ownership” (per process) → no communication → easy to write parallel programs

Disadvantage: Harder to manage data locality (may be *beyond* average user’s control)

Implementations

- Stand-alone **shared memory** machines: supported by native OS, compilers, actor models; e.g., POSIX API, UNIX’s shared mem segments (shmget, shmat, shmctl)
- Distributed-shared memory** machines: Partitioned Global Address Space (PGAS) libraries and HW support

Note: Not covered further in this lecture series

Parallel Computers and Their Programming

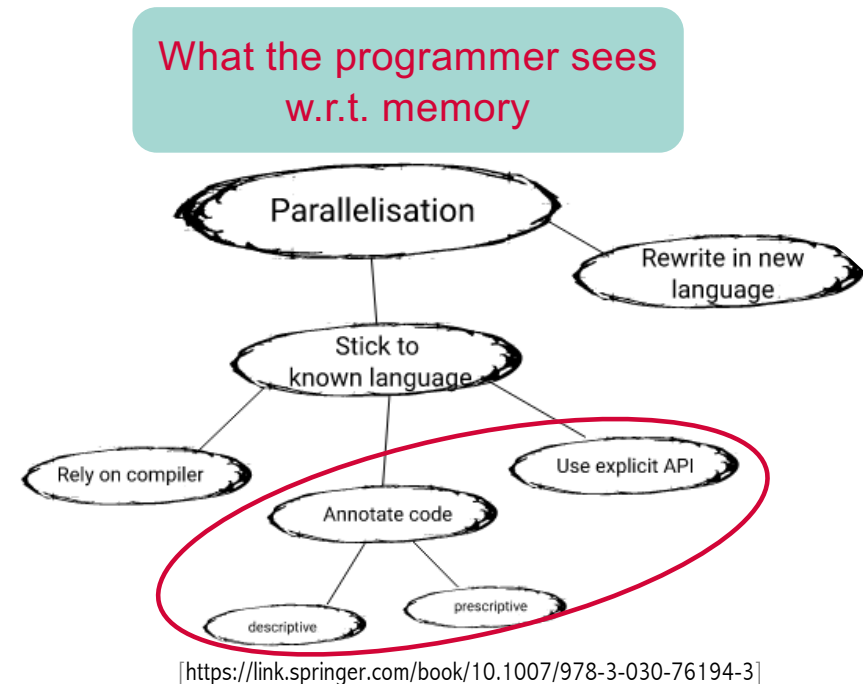
As Easy as A, B, C

A. Memory architectures

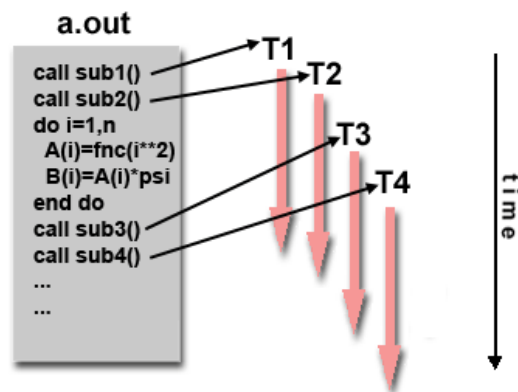
B. Parallel programming models

1. Shared memory (**without** threads)
2. **Shared memory** (**with** threads)
3. **Distributed memory** / Message Passing
4. Data parallel
5. Hybrid **distributed shared** memory
6. Single Program Multiple Data (SPMD)
7. Multiple Program Multiple Data (MPMD)

C. Network topologies



Programming Model 2: Shared memory (**with** threads)

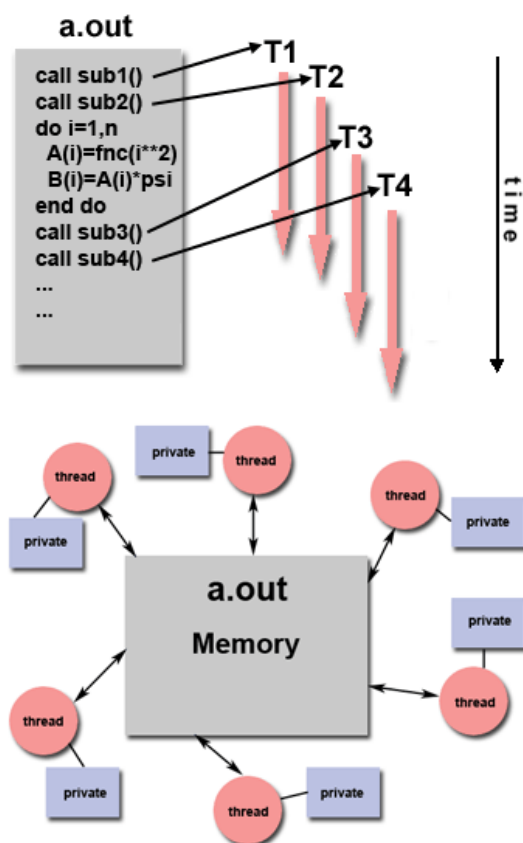


- A **single** "heavy weight" process (**a.out**) can have **multiple** "light weight" **concurrent** execution paths (threads T_i)
- Each thread has local data, but also, **shares** the entire resources of **a.out**, including global memory
- Threads 'communicate' via **global memory** (updating address locations)

Advantage: *Implicit* parallelism, not too hard to expose/express

Disadvantage: (Still) manual parallelization

Programming Model 2: Shared memory (with threads)



- A **single** "heavy weight" process (**a.out**) can have **multiple** "light weight" **concurrent** execution paths (threads T_i)
- Each thread has local data, but also, **shares** the entire resources of **a.out**, including global memory
- Threads 'communicate' via **global memory** (updating address locations)

Advantage: *Implicit* parallelism, not too hard to expose/express

Disadvantage: (Still) manual parallelization

Implementations

- POSIX Threads: IEEE POSIX 1003.1c standard (1995); lib-based, in Linux
- **OpenMP (Open MultiProcessing)**: Industry standard (1997+); **C/C++**, **FTN**
- Microsoft, Java, **Python**, **CUDA**, **HIP**, etc.
- *Note: OpenMP Introduction in Chapter 5.3a*

Parallel Computers and Their Programming

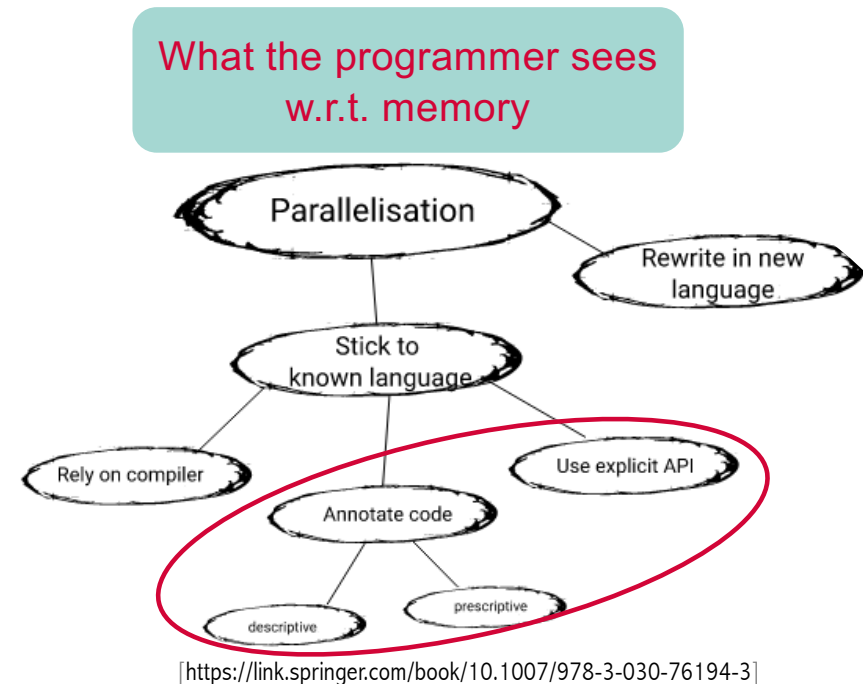
As Easy as A, B, C

A. Memory architectures

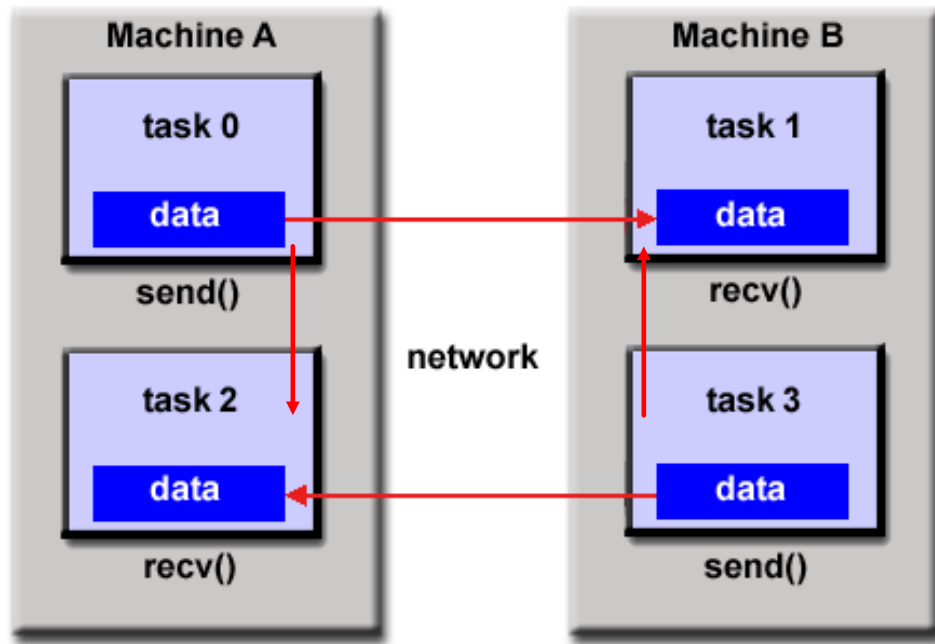
B. Parallel programming models

1. Shared memory (without threads)
2. Shared memory (with threads)
3. Distributed memory / Message Passing
4. Data parallel
5. Hybrid distributed shared memory
6. Single Program Multiple Data (SPMD)
7. Multiple Program Multiple Data (MPMD)

C. Network topologies



Programming Model 3: Distributed memory / Message Passing

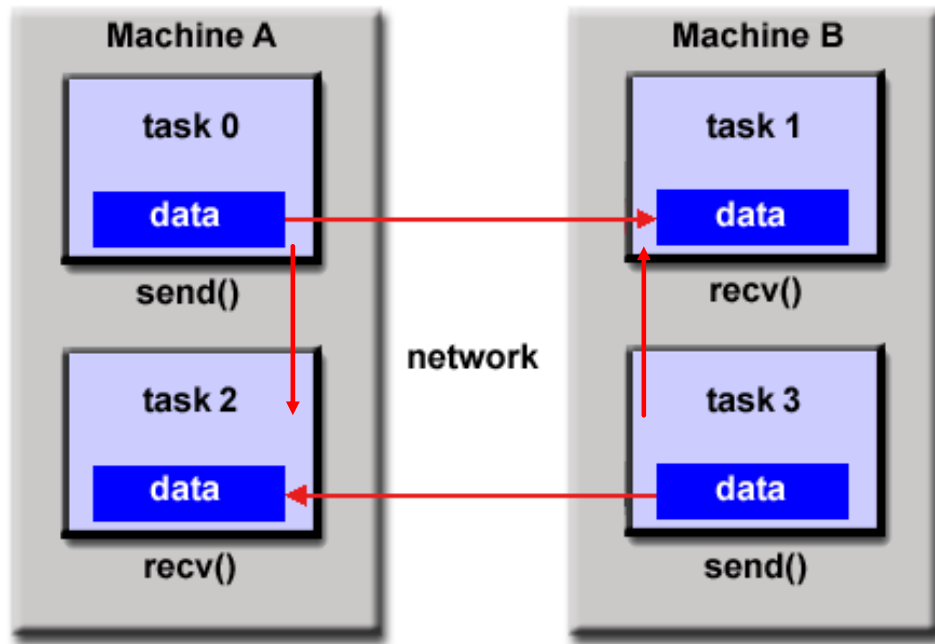


- 'Tasks' (processes) use their own local memory during computation
- 'Tasks' reside on the **same** physical machine and/or **across other machines**
- 'Tasks' **communicate** via **sending** and **receiving** messages

Advantage: Scales (well) beyond single machines

Disadvantage: *Explicit* parallelism; (still) manual parallelization

Programming Model 3: Distributed memory / Message Passing



- 'Tasks' (processes) use their own local memory during computation
- 'Tasks' reside on the **same** physical machine and/or **across other machines**
- 'Tasks' **communicate** via **sending** and **receiving** messages

Advantage: Scales (well) beyond single machines

Disadvantage: *Explicit* parallelism; (still) manual parallelization

Implementations

- **MPI (Message Passing Interface):**
 - Forum since 1992, portability
 - Industry standard: MPI-1 1994, MPI-2 1996, MPI-3 2012, MPI-4 2020*
- PVM (Parallel Virtual Machine) < 1990s
- **Note:** MPI introduction in Chapter 5.3b

Parallel Computers and Their Programming

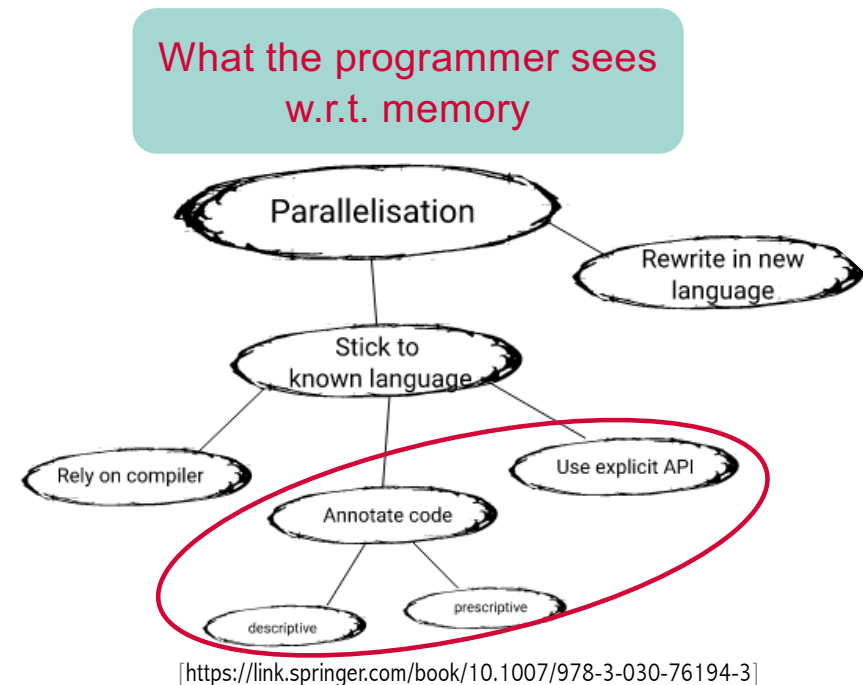
As Easy as A, B, C

A. Memory architectures

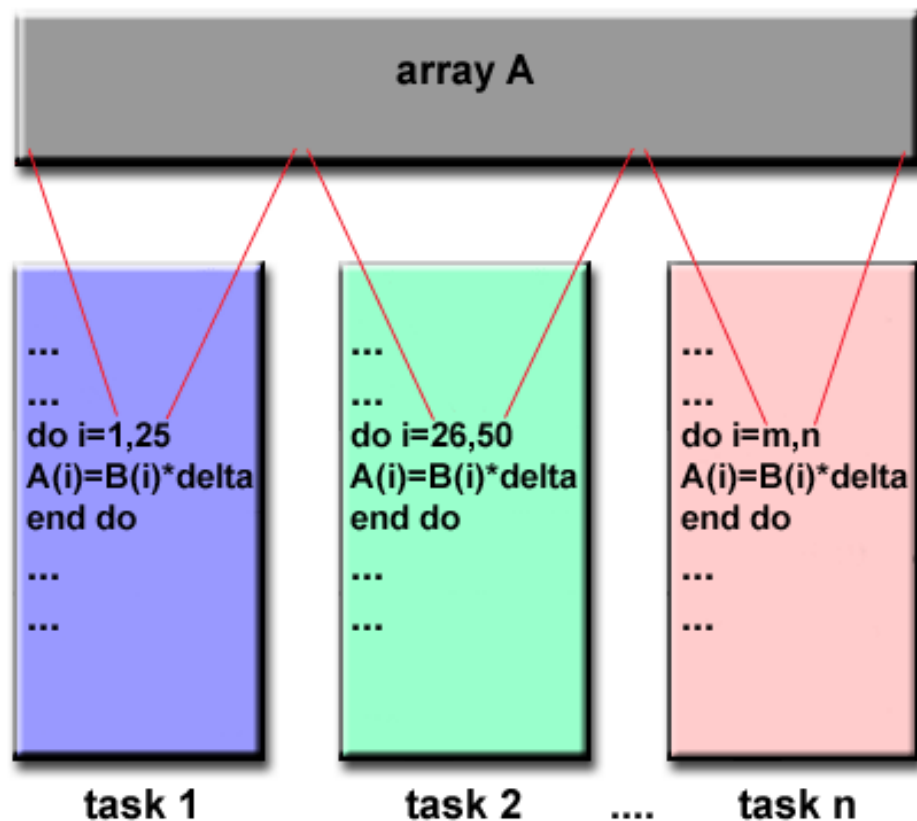
B. Parallel programming models

1. Shared memory (**without** threads)
2. **Shared memory** (**with** threads)
3. **Distributed memory** / Message Passing
4. **Data parallel**
5. Hybrid **distributed shared** memory
6. Single Program Multiple Data (SPMD)
7. Multiple Program Multiple Data (MPMD)

C. Network topologies



Programming Model 4: Data Parallel / **Partitioned** Global Address Space (**PGAS**)



- Address space treated **globally**
 - Locally / on same machine: **shared memory**
 - Globally / across different machines: **PGAS**
- **Parallel work**: operations on a data set (data structure: 1-2D array/cube)
- Tasks work **collectively** on the **same data** structure, but **independently** on a **different partition** of the same data structure (**SIMD**)

Implementations

- **Shared memory**: all tasks access work data structure via global memory (e.g., **OpenMP**)
- **Distributed memory**: global data structure may be split up logically/physically among tasks => PGAS
 - **Synchronous** PGAS: Coarray Fortran, Unified Parallel C, Global Arrays
 - **Asynchronous** PGAS (tasks): HPX, X10, Chapel, Charm++
 - **Note**: Not covered further in these lectures

Parallel Computers and Their Programming

As Easy as A, B, C

A. Memory architectures

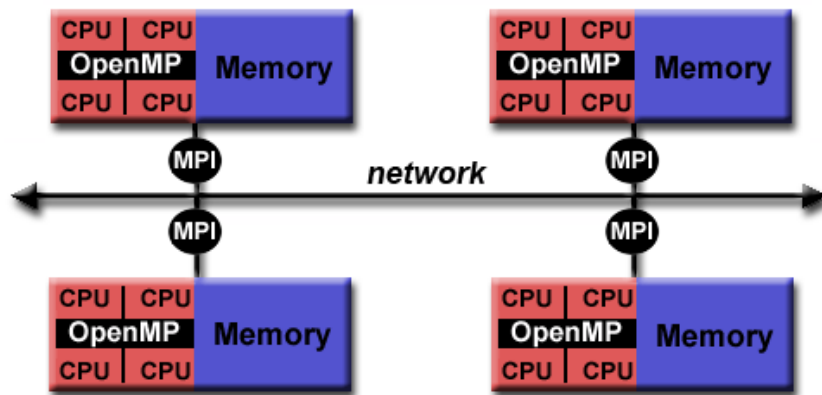
B. Parallel programming models

1. Shared memory (**without** threads)
2. **Shared memory** (**with** threads)
3. **Distributed memory** / Message Passing
4. Data parallel
5. Hybrid **distributed shared** memory
6. Single Program Multiple Data (SPMD)
7. Multiple Program Multiple Data (MPMD)

C. Network topologies

What the programmer sees
w.r.t. memory

Programming Model 5: Hybrid **Distributed Shared** Memory



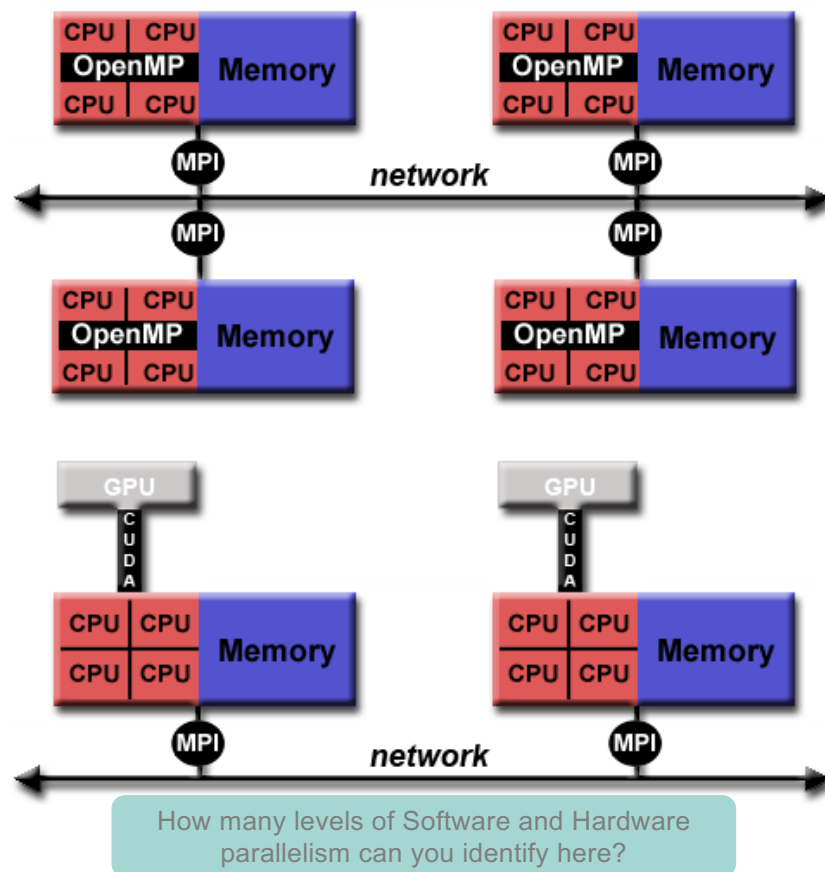
Message passing + Shared memory: **MPI+OpenMP**

- MPI processes: 1 computationally-intensive process per node/UMA domain + inter-process *communication* (IPC) across nodes over the network
- OpenMP threads: execute computationally-intensive *kernels* + on-node local data
- **Advantage**: Suitable for most popular clustered **multi/many-core** machines

Programming Model 5: Hybrid Distributed Shared Memory

IN FDS
LECTURE

MORE IN HPC
LECTURE



Message passing + Shared memory: MPI+OpenMP

- MPI processes: 1 computationally-intensive process per node/UMA domain + inter-process *communication* (IPC) across nodes over the network
- OpenMP threads: execute computationally-intensive *kernels* + on-node local data
- Advantage: Suitable for most popular clustered multi/many-core machines

Message passing + Shared memory: MPI+CUDA

- MPI processes: 1 MPI process per node/UMA domain + local memory
- CUDA threads: Several CUDA threads executing computationally-intensive *kernels* on GPUs + manage exchanges between CPU memory (host) to GPU memory (device)
- Other MPI+X or Y instances
 - 1 MPI process on a node + multiple Pthreads on CPUs of the node
 - 1 MPI process on a node + multiple threads on non-GPU accelerators (e.g., TPUs, VPUs, DSPs, FPGAs)
- Note: Covered briefly in Chapter 5.3 but in greater depth in FS26 HPC lecture

Parallel Computers and Their Programming

As Easy as A, B, C

A. Memory architectures

B. Parallel programming models

1. Shared memory (**without** threads)
2. **Shared memory** (**with** threads)
3. **Distributed memory** / Message Passing
4. Data parallel
5. Hybrid **distributed shared** memory
6. Single Program Multiple Data (SPMD)
7. Multiple Program Multiple Data (MPMD)

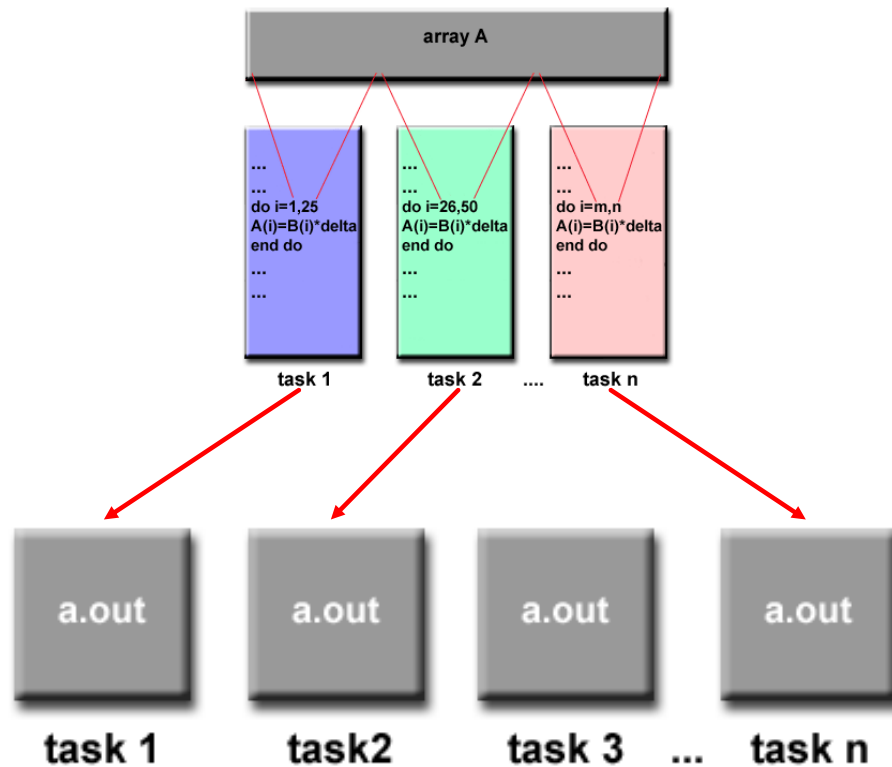
What the programmer sees
w.r.t. memory

C. Network topologies

*Every programming model in this course basically boils down to SPMD.
We never write two different programs that collaborate.*

*Even higher abstraction levels can be built upon any combination of
programming models 2-5.*

Programming Model 6: Single Program Multiple Data (SPMD)



Note: By default, covered in this course
& in greater depth in HPC Spring lecture

- **Single Program:** All tasks execute **simultaneously** a **copy of the same** program
 - Program: tasks, threads (shared-mem), processes (message passing), or hybrid
- **Multiple Data:** All **tasks** may use **different** data subset
- **SPMD** programs have *logic programmed* to allow different 'tasks' to execute only those parts of the program they are designed to execute (e.g., if rank=0, read data)

Advantage: Most suited for *data decomposition* (Step 1)

Most used parallel programming model for **multi-node computing clusters** or multi-device nodes, using

- PM2. **Shared memory** (with threads)
- PM3. **Distributed memory** / Message Passing
- PM5. Hybrid **distributed shared** memory

Parallel Computers and Their Programming

As Easy as A, B, C

A. Memory architectures

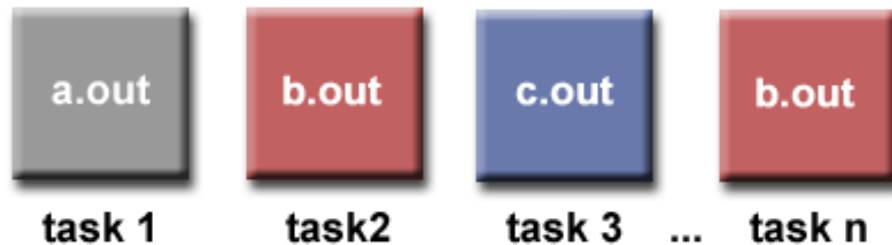
B. Parallel programming models

1. Shared memory (**without** threads)
2. **Shared memory** (**with** threads)
3. **Distributed memory** / Message Passing
4. Data parallel
5. Hybrid **distributed shared** memory
6. Single Program Multiple Data (SPMD)
7. **Multiple Program Multiple Data (MPMD)**

C. Network topologies

What the programmer sees
w.r.t. memory

Programming Model 7: Multiple Program Multiple Data (MPMD)



- **Multiple Programs:** 'Tasks' may execute **different** programs **simultaneously**
 - Program: tasks, threads (shared-mem), processes (message passing), or hybrid
- **Multiple Data:** All **tasks** may use **different** data
- **MPMD** applications are *less common* than SPMD ones
 - But can be thought of as application workflows

Advantage: Most suited for *functional decomposition* (Step 1): where one task → one program

Note: Not explicitly covered in this lecture series but all SPMD/SIMD concepts implicitly apply to each task.

What Have We Learned Today?

A. Memory architectures

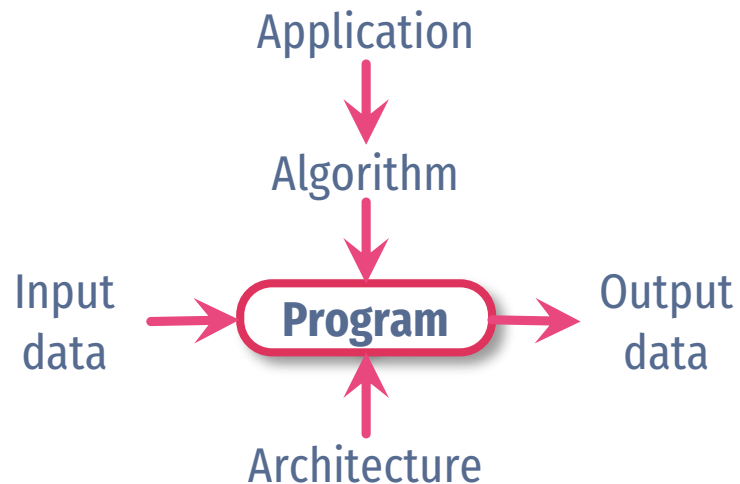
B. Parallel programming models

1. Shared memory (**without** threads)
2. **Shared memory** (**with** threads)
3. **Distributed memory** / Message Passing
4. Data parallel
5. Hybrid **distributed shared** memory
6. Single Program Multiple Data (SPMD)
7. Multiple Program Multiple Data (MPMD)

C. Network topologies

What the programmer sees
w.r.t. memory

What Have We Learned Today?



Parallel Program = Parallel Algorithms + Parallel Data Structuring (+ Parallel Hardware View)

a Memory Architectures

b **Parallel Programming Models**

c Network Topologies

Summary

- **Programming models** (PM) are an abstraction above hardware and memory architectures, offer a view of **data** (where stored, how organized) & **execution** (what the program does)
- **Programs = algorithms + data structuring** (+ hardware view)*
- Where **architecture** meets **applications** and vice versa
 - 👉 Cannot write **parallel** programs *without a faint idea* of the potential **parallel** system
- **Most common parallel programming models**
 - PM2. **Shared memory** (with threads)
 - PM3. **Distributed memory** / Message Passing
 - PM5. Hybrid **distributed shared** memory